

- $G_1 = (N, E, P, S)_{\text{sentence}} \quad P: \alpha \rightarrow \beta$

Date, 25. 10. 23

$S \rightarrow NP \quad VP$

$NP \rightarrow N$

$NP \rightarrow \text{Adj } N$

$NP \rightarrow \text{Art } \text{Art } N$

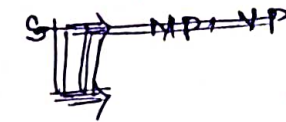
$N \rightarrow \text{Lipsa}$

$\text{Adj} \rightarrow \text{Punctual}$

$VP \rightarrow V$

$VP \rightarrow V \text{ Adv}$

$VP \rightarrow V \text{ NP}$



$NP, VP, N, \text{Adj} \rightarrow$ Non-terminals
(Begin with with capital letter)

Eg:

Khushi smiles always.

$S \Rightarrow NP \quad VP$

$\Rightarrow N \quad VP$

$\Rightarrow \text{Khushi} \quad VP$

$\Rightarrow \text{Khushi} \quad V$

$\Rightarrow \text{Khushi} \text{ smiles}$

Q. How to identify a context free language in a given grammar

$\rightarrow$ ~~Each~~ In a context free grammar, each and every production rules have one ~~to~~ non-terminal in L.H.S.

$\rightarrow$ For representing alternate don't use / (slash symbol). Use $\rightarrow$ |

Eg:- $\text{Adj} \rightarrow \text{punctual} \mid \text{careless}$

$\rightarrow$ This gives an advantage of deriving valid string of a language.

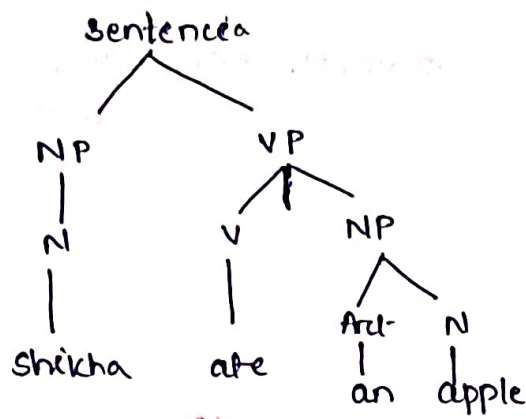
Eg:- $w_1 = \text{Khushi smiled}$

$w_2 = \text{shikha ate an apple}$
(T) (T) (T) (T)

$V \rightarrow \text{came} \mid \text{ate} \mid \text{smiled}$

$\text{Art} \rightarrow \text{a} \mid \text{an} \mid \text{the}$

$N \rightarrow \text{shalu} \mid \text{raja} \mid \text{Khushi} \mid$
 $\text{apple} \mid \text{shikha}$



→ Tree line structure is used to represent string.

(Derivation Tree)

• $G = (N, \Sigma, P, E)$

$\phi: \alpha \rightarrow \beta$

$\alpha \in N$

$\Sigma = \{a, +, *, (,)\}$ [set of Terminals]

Expression: $\downarrow$

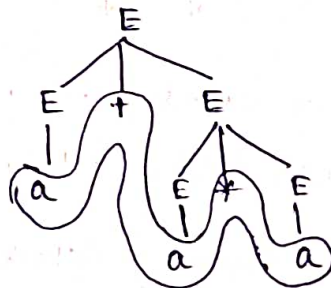
$a + (a * a)$
 $(a * a) + (a * a)$

Rules

- 1) $E \rightarrow E + E$
- 2) $E \rightarrow E * E$
- 3) $E \rightarrow a$

→

for deriving this $\downarrow$

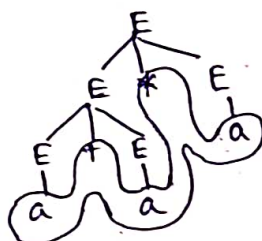


(I)

→ Derivation tree is a tree with nodes are labelled with terminals & non-terminals, root is start symbol, branches are as per RHS of production rule with LHS being the root of the subtree

→

int a = 5;



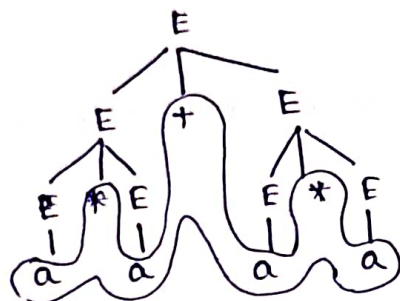
(II)

→ There is no unique derivation tree for a string.

→ [&] makes an ambiguous situation]

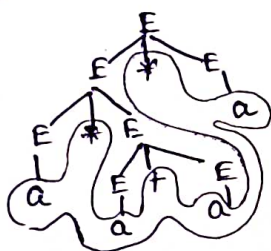
[4-different valid tree]

(ii) $a * a + a * a$



1, 2, 2, 3, 3, 3, 3

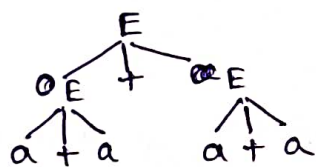
2, 3, 1, 3, 2, 3, 3



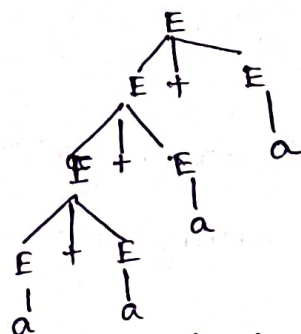
⇒ If the derivation tree for all string or atleast one string are not unique, then the grammar is said to be ambiguous

→ Any string having more than one operator and production rule have same non terminals having both (all operator) then the derivation tree will not be unique.

Eg: $a + a + a + a$



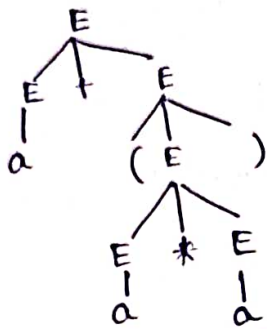
(I)



(II)

eg: $a + (a * a)$

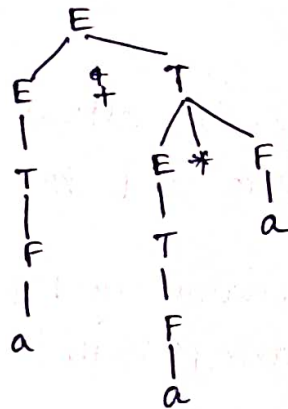
1. $E \rightarrow E + E$
2. $E \rightarrow E * E$
3. $E \rightarrow a$
4. $E \rightarrow (E)$



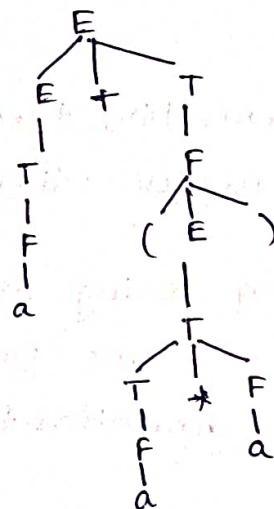
→ To ~~remove~~ ^{eliminate} ambiguity, let's introduce more than one non-terminal

1. $E \rightarrow E + T \mid T$
2. $T \rightarrow T * F \mid F$
3. $F \rightarrow (E) \mid a$

Eg: $a + a * a$



$a + (a * a)$



→ Language of arithmetic expression not inherently ambiguous.

→ If each grammar is ambiguous then language is inherently ambiguous.